

Where the First Root Comes From

A section-by-section guide to `root_origins_study.m`:
the cold lottery, the two-half ladder, the covector handoff, and the basin hunt

M. Casey (Coorbital, `orbit_transfer/DR0_tulip`)

2026-09-16

Abstract

Every seed in the costate library is a previous root. So where did the first one come from? `root_origins_study.m` answers that by doing it: it starts from nothing but two periodic orbits and an engine, and measures the three mechanisms that produce a first root — a cold direct solve where the problem is easy, a ladder down the engine axis that changes chart from collocation to shooting halfway, and a branch-blind hunt for other families. This guide walks the script section by section. Each section is explained three times: as a picture (ELI5), as the reasoning needed to trust it (Intuition), and as the equations and the code (Rigor), with the code snippets sliced verbatim from the script at build time so they cannot drift from it, and the campaign’s measurements called out where they settled a question. It ends with what the script does *not* establish, because the two external reviews of it (FINDINGS 76–77) were mostly about sentences that claimed more than the computation earned. Companions: `seeds_and_solution_route.tex` (what a seed is), `transfer_study_guide.tex` (the gate stack the certifier runs), `process/ROOT_ORIGINS_STUDY.md` (how to run it).

Contents

1	The question, and the honest answer first	1
2	The flow of the script	3
3	§0 — User inputs and the units	3
4	§1 — The two orbits and the two endpoints	6
5	§2 — The cold lottery	6
6	§3 — The direct ladder	8
7	§4 — The handoff: where costates come from	10
8	§5 — The indirect ladder	13
9	§6 — Certification, and adopting the polished root	15
10	§7 — The basin hunt	16
11	§8 — The self-check: gates and outcomes are different things	18

12 What the script does not establish	19
13 Where these caveats came from	19
14 Summary	20

1 The question, and the honest answer first

ELI5

Imagine a library of maps for flying a small spacecraft from one orbit around the Moon to another. Every map in the library was drawn by starting from a neighbouring map and nudging it. Fine — but somebody had to draw the first map with no neighbour to copy. How?

For this particular library the honest answer is: the first map was a gift. A converged transfer for exactly this cell arrived with the `pumpkynPie` package, and every early measurement in the campaign is quoted against it. This script is what you do for every other cell, thrust and orbit pair — which is nearly everything — where there is no gift.

Intuition

Three mechanisms, in the order a campaign uses them, and each one is a *measurement* in the script rather than a claim:

1. **The cold lottery** (Section 5). Solve the operating point cold, from nothing but the endpoints and a time-of-flight guess, at several mesh densities. The accepted solves do not agree. That motivates informed seeding or continuation rather than trusting one cold solve.
2. **The ladder** (Section 6–8). Start where the problem is easy — 15 N, a half-day transfer under one turn, converges cold in seconds — and walk the engine down, each rung seeded by the one above. The walk has two halves and a seam: direct collocation down to 0.5 N, then a covector harvest, then multiple shooting the rest of the way.
3. **The basin hunt** (Section 10). From the root reached, ask the direct solver for other arrival phases. It is branch-blind: it lands wherever its warm start is near. That is a hazard if you assume it returns your family, and a discovery tool if you do not.

Rigor

The problem throughout is the Earth–Moon CR3BP in rotating barycentric nondimensional units ($\mu^* = 0.012150585609624$, $l^* = 389703.26$ km, $t^* = 382981.29$ s), minimum time, low thrust, free t_f , fixed endpoints:

$$\dot{\mathbf{r}} = \mathbf{v}, \quad \dot{\mathbf{v}} = \mathbf{g}(\mathbf{r}, \mathbf{v}) + \frac{T_{\text{nd}} q}{m} \mathbf{u}, \quad \|\mathbf{u}\| = 1, \quad q \in [0, 1], \quad \dot{m} = -\frac{T_{\text{nd}} q}{c_{\text{nd}}},$$

with m a mass *fraction* ($m(0) = 1$), $\mathbf{r}(0), \mathbf{v}(0)$ on the DRO at phase s_D and $\mathbf{r}(t_f), \mathbf{v}(t_f)$ on the tulip at phase s_A . The Hamiltonian, adjoints and the two facts the script leans on:

$$H = 1 + \lambda_r^\top \mathbf{v} + \lambda_v^\top \mathbf{g} + T_{\text{nd}} q \left(\frac{\lambda_v^\top \mathbf{u}}{m} - \frac{\lambda_m}{c_{\text{nd}}} \right), \quad \mathbf{u}^* = -\frac{\lambda_v}{\|\lambda_v\|}, \quad \dot{\lambda}_m = -\frac{\partial H}{\partial m} = -\frac{T_{\text{nd}} q \|\lambda_v\|}{m^2}.$$

This is the normal, unconstrained minimum-principle formulation ($H = L + \lambda^\top \mathbf{f}$ with $L = 1$, controls minimising H), with positive mass, no terminal-mass penalty or active terminal-mass bound, and endpoint states that do not depend on t_f . Under those assumptions the terminal mass is free, so $\lambda_m(t_f) = 0$ (fixed terminal position and velocity do *not* make their costates vanish), and integrating backward,

$$\lambda_m(t) = \int_t^{t_f} \frac{T_{\text{nd}} q(\xi) \|\lambda_v(\xi)\|}{m(\xi)^2} d\xi \geq 0.$$

After minimising over the direction, $\partial H / \partial q = -T_{\text{nd}} (\|\lambda_v\|/m + \lambda_m/c_{\text{nd}})$, strictly negative wherever $\|\lambda_v\|/m + \lambda_m/c_{\text{nd}} > 0$; minimising H then selects $q = 1$. A nonzero primer is sufficient for that, not necessary.

Why primer zeros cannot spoil it. The adjoints of $\mathbf{r}$ and $\mathbf{v}$ are linear and homogeneous in (λ_r, λ_v) : $\dot{\lambda}_r = -\mathbf{g}_r^\top \lambda_v$, $\dot{\lambda}_v = -\lambda_r - \mathbf{g}_v^\top \lambda_v$. At a primer zero, $\dot{\lambda}_v = -\lambda_r$. If λ_r vanished there too, both would vanish identically; then $\dot{\lambda}_m = 0$ and $\lambda_m \equiv \lambda_m(t_f) = 0$, giving $H \equiv 1$, which contradicts $H(t_f) = 0$. So primer zeros are isolated, $q = 1$ almost everywhere, and the direction's being undefined at isolated instants does not change the trajectory: **minimum time is all-burn** under these assumptions. An *active* lunar-clearance constraint would need the constrained minimum principle instead; the shooter uses the unconstrained one.

The shooting unknown is $z = (\lambda(0), t_f) \in \mathbb{R}^8$, with $H(t_f) = 0$ closing the free-time problem.

2 The flow of the script

A note on numbering. “Script § n ” and the § n in the section headings and the diagram are the script’s own section numbers, 0 to 8; “Section n ” is this document’s. They differ by three.

Intuition

Two things to notice in the diagram. The data objects change type at script §4: above it the trajectory is a discrete collocation solution ($\mathbf{X}$, $\mathbf{U}$, and the NLP’s multipliers); below it, it is a shooting root (z plus junction states). Script §4 is the first place the multipliers are *mapped* into a state-and-costate shooting seed; script §7 harvests again, from each accepted probe. And the record $\mathbf{S}$ grows as the run goes: the script checkpoints each completed mesh, rung, handoff, certification and probe, so an hour-long run can be read while it is still going. A stage interrupted mid-way may be absent, the cold and direct records keep diagnostics rather than full solution arrays, and a failed save warns rather than guaranteeing anything.

3 §0 — User inputs and the units

ELI5

Everything the run depends on sits in one place at the top: which two orbits, which point on each, what engine we are trying to reach, the two ladders that get there, where to hunt, how long each step may take, and how close is close enough. Change the experiment here and nowhere else.

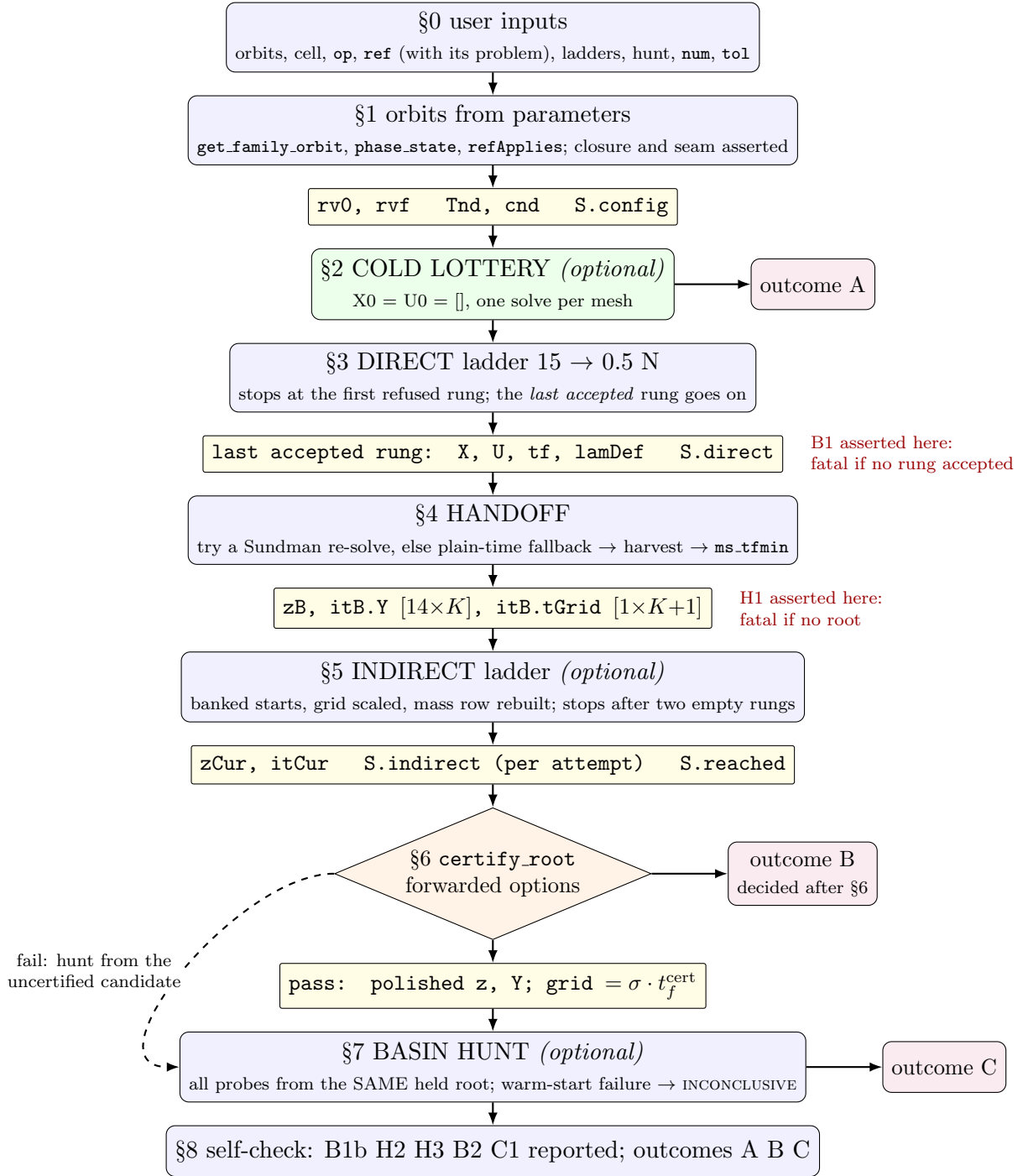


Figure 1: Execution order, top to bottom; the § numbers are the script’s own. Yellow boxes are the objects handed on. The two red notes are the named fatal checks, asserted where their stage ends — other validation assertions (orbit closure, usable multipliers, a malformed passing certificate) can also abort. Optional stages can be switched off in script §0; outcome B is decided only after certification, because “reached the target engine” and “holds a certified root there” are different claims.

Intuition

The reference deserves a word. The campaign's certified root at this cell is stored with the *problem it belongs to* — orbits, phases, engine, mass — and reference comparisons are computed only when the live problem matches. Otherwise the script says the reference belongs elsewhere; the cold table keeps its “vs ref” column but shows NaN in it, and the summary and the certified-root comparison suppress the number. Without that guard, editing the cell would leave a figure on screen labelled “the campaign's answer here” that belongs to somewhere else.

section 0: the operating point

```
op = struct( ...
    'thrustN', 0.070, ...    % N -- the abstract's Hall thruster
    'ispS',    900, ...    % s
    'm0kg',    150);      % kg
```

section 0: the reference carries its problem

```
ref = struct('tfDays', 17.7976, 'tau', 1.0, 'Np', 7, 'pm', -1, 'sD', 0.0, 'sA', 0.0754, ...
    'thrustN', 0.070, 'ispS', 900, 'm0kg', 150, ...
    'what', 'the fast-family anchor of the 70 mN library (arrived with pumpkynPie)');
```

Rigor

Nondimensionalisation. With $g_0 = 9.80665 \text{ m/s}^2$ and the CR3BP units,

$$T_{\text{nd}} = \frac{T}{m_0} \frac{t^{\star 2}}{1000 l^{\star}}, \quad c_{\text{nd}} = \frac{I_{\text{sp}}}{t^{\star}} g_{0,\text{nd}}, \quad g_{0,\text{nd}} = g_0 \frac{t^{\star 2}}{1000 l^{\star}},$$

so T_{nd} is the thrust acceleration at unit mass fraction and c_{nd} the exhaust speed, both in ND. The factor 1000 converts $l^{\star}$ from km to m. The tolerance block keeps two kinds of number apart on purpose: what a solver is *asked* for (`Rsolve`, 10^{-11}) and what a gate *accepts* (R , 10^{-8}). A solve that plateaus above its own target but inside the gate is a root.

section 0: the ND conversions

```
g0 = 9.80665*tStar^2/(1000*lStar);
ndT = @(TN, m) (TN/m)*tStar^2/(lStar*1000);    % N -> ND thrust acceleration at m = 1
ndC = @(isp) (isp/tStar)*g0;                   % s -> ND exhaust speed
```

section 0: acceptance thresholds and solve targets

```
tol = struct( ...
    'closure', 1e-7, ...    % orbit periodicity
    'seamDeriv', 1e-6, ... % interpolant derivative mismatch across s = 0
    'defect', 1e-8, ...    % collocation defect on an accepted rung
    'interp', 1e-8, ...    % Hermite-Simpson interpolation residual (NOT in maxDefect)
    'tfSpread', 1e-8, ...  % lifted-time continuity across the nodes
    'termErr', 1e-7, ...   % terminal boundary residual
    'unit', 1e-8, ...     % |u| = 1 on the solved directions
    'thrMin', 1e-3, ...   % 1 - throttle: the throttle is FREE, so saturation is CHECKED (see above)
    'R', 1e-8, ...        % multiple-shooting residual, inf-norm (acceptance)
```

```
'Rsolve',    1e-11, ... % what the shooting solver is ASKED for (tighter than the gate)
'tfCluster', 0.02, ... % ND-relative spread within which two t_f are one cluster
'same',      1e-2);    % |t_f - the reference's|/t_f to call the flight times equal
```

Where it breaks / what is assumed

On **thrMin**: the direct solver leaves the throttle free *by design*, so that saturation can be checked rather than assumed. An interior-point solver leaves a barrier slack on every bound — measured 4×10^{-8} to 1.3×10^{-6} at IPOPT tolerance 10^{-7} , and the first full run tripped a 10^{-6} gate on the 15N rung at $N = 800$ with 1.3×10^{-6} . A real switch takes the throttle to 0. The 10^{-3} gate sits between. It is a defensible *seed-compatibility screen*, not a proof of saturation: its inequalities are strict, so it accepts a sustained deficit just below 0.1%, and a dip between the sampled stations can escape it. The all-burn shoot and the certificate downstream are what carry the claim.

4 §1 — The two orbits and the two endpoints

ELI5

Both orbits are rebuilt from their own parameters every run — the DRO from its period, the tulip from its petal count and branch — and a point on each is picked by a fraction of a period. No transfer is loaded anywhere; that is the whole point of the script.

Intuition

Closure alone is not enough. An orbit can close to 10^{-7} and still have an interpolant whose *slope* jumps where phase wraps from 1 back to 0. That matters here because phase is a continuation coordinate: nearby phases seed related solves, and a kink at the wrap would make two neighbouring endpoints disagree about the direction the orbit is moving. So the smoothness across the seam is checked, not just the closure.

Rigor

`get_family_orbit` starts from a catalogued family seed and refines it to closure; the script requires both sampled closures $\|\mathbf{x}(T) - \mathbf{x}(0)\| < 10^{-7}$. `phase_state` returns a periodic interpolant $s \mapsto \mathbf{x}(s)$ on $[0, 1)$ with its seam diagnostics; the script gates the larger seam-*derivative* mismatch below 10^{-6} and does not separately gate the seam value. The endpoints are $\mathbf{x}_0 = \mathbf{x}_D(s_D)$, $\mathbf{x}_f = \mathbf{x}_A(s_A)$.

section 1: endpoints from phase fractions

```
[stateD, seamD] = phase_state(tD, rvD);
[stateA, seamA] = phase_state(tT, rvT);
rv0 = stateD(phase.sD);    rv0 = rv0(1:6);
rvf = stateA(phase.sA);
```

5 §2 — The cold lottery

ELI5

Ask the big optimiser to solve the real problem with no hints at all: just the two endpoints, a guess at how long the trip takes, and how finely to chop time. Do it three times, chopping time into 400, 800 and 1600 pieces. Two of the three pass every acceptance check; the third stops at its iteration limit. The two that pass give different trips, and nothing on screen tells you which one you would have wanted.

Intuition

The direct method needs no previous converged transfer. With $\mathbf{x}_0 = \mathbf{u}_0 = []$ it still builds an initial numerical guess of its own — states linear between the endpoints, a mass ramp, thrust along the chord — and IPOPT takes it from there. Both things matter: whether it converges, and *what* it converges to. Minimum-time extremals of this problem come in families, and a cold solve lands wherever its own discretisation happens to lead. Only the mesh changes between the three runs; the answers change with it. That motivates informed seeding or continuation on a catalog pair rather than trusting one cold solve.

What the lottery does *not* show, and the script now says so: that each converged row is a distinct continuous extremal. Changing N changes the finite NLP, its internal guess and its conditioning; a flight-time disagreement alone does not separate “different physical extrema” from those effects. It is mesh sensitivity of the accepted *discrete* solutions — which is enough for the point being made.

Rigor

The transcription is Hermite–Simpson with N intervals on normalised time $s \in [0, 1]$, $h = 1/N$. The flight time is *lifted* to a per-node variable $\mathcal{T}_j$ with continuity $\mathcal{T}_{k+1} = \mathcal{T}_k$, and the normalised-time right-hand side is

$$\mathbf{F}_j = \mathcal{T}_j \mathbf{f}(\mathbf{x}_j, \mathbf{u}_j, q_j), \quad \frac{d\mathbf{x}}{ds} = t_f \mathbf{f}.$$

The defect and the midpoint interpolation are then

$$\begin{aligned} \mathbf{x}_{k+1} - \mathbf{x}_k - \frac{h}{6}(\mathbf{F}_k + 4\mathbf{F}_{k+\frac{1}{2}} + \mathbf{F}_{k+1}) &= 0, \\ \mathbf{x}_{k+\frac{1}{2}} - \frac{\mathbf{x}_k + \mathbf{x}_{k+1}}{2} - \frac{h}{8}(\mathbf{F}_k - \mathbf{F}_{k+1}) &= 0, \end{aligned}$$

with $\mathcal{T}_{k+\frac{1}{2}} = (\mathcal{T}_k + \mathcal{T}_{k+1})/2$; at feasibility every $\mathcal{T}_j = t_f$. Equivalently, Hermite–Simpson applied to the augmented state $(\mathbf{x}, \mathcal{T})$ with derivative $(\mathcal{T}\mathbf{f}, 0)$. Lifting keeps each stencil local — the nodes, the midpoint and the local $\mathcal{T}$ of one interval — so the KKT matrix stays banded; a single scalar t_f would add a dense bordering column (the solver’s own header records MATLAB dying on exactly that at $N = 400$, an implementation observation rather than a mathematical consequence). The objective is t_f ; the throttle is a free variable in $[0, 1]$. Every cold solve is judged by the same *core* predicate as every other direct solve in the script (`directOK`, Section 6); a run that hit its iteration limit is printed as an *iterate* and excluded.

On the accepted rows the script reports the spread and a cluster count:

$$n_{\text{cluster}} = |\text{uniquetol}(\{t_f^{(i)}\}, \epsilon)|, \quad \epsilon = \text{tol.tfCluster} \times \min_i t_f^{(i)}, \text{ absolute,}$$

an explicit absolute tolerance tied to the shortest accepted flight time (`uniquetol`'s default scales by the largest element, so a long outlier would regroup the others). A cluster of flight times is not a basin — two roots can share a t_f and one family can span several — it is the cheap first read of the table.

section 2: a genuinely cold solve

```
oM = solveDirect(rv0, rvf, TndOp, cndOp, muStar, N, [], [], cold.tf0, dCold);
[okM, whyM] = directOK(oM, tol);
```

section 2: the statistics on the accepted rows

```
spread = (max([good.tf]) - min([good.tf]))/min([good.tf]);
nCluster = numel(uniquetol([good.tf], tol.tfCluster*min([good.tf]), 'DataScale', 1));
below = nnz([good.nodeRadiusKm] < num.clearKm);
agree = nnz(abs([good.rel]) < tol.same); % 0 when the reference does not apply
```

Measured

Reference at this cell: $t_f = 4.0152 \text{ ND} = 17.798 \text{ d}$. Cold, unconstrained, same solver, same t_f guess (4.0 ND), only N varying.

N	t_f (ND)	vs. reference	min node radius (km)	status
<i>2026-08-03 (logged as altitude; +1737.4 here)</i>				
400	4.3807	+9.1%	6556	converged
800	4.4507	+10.8%	1394	converged, through the Moon
1600	4.7833	+19.1%	6417	converged, 7.8 m worst-interval error
<i>2026-09-16, this script</i>				
400	4.6809	+16.6%	6563	converged
800	4.9909	+24.3%	2243	iterate: iteration limit, excluded
1600	6.4126	+59.7%	1995	converged

Spread 37%, two clusters, none within 1% of the reference. Outcome A: SUPPORTED. “Min node radius” is over the collocation nodes — a screen, not a certified periselene.

6 §3 — The direct ladder

ELI5

Start with an engine so strong the trip takes half a day and under one turn around the Moon — the optimiser finds that one cold in seconds. Then turn the engine down a little, hand the optimiser the last trip as its starting point, and solve again. Repeat. A good starting point usually makes each solve easy; it does not guarantee the optimiser stays on the same

family, which is why every step is checked.

Intuition

Three things make the ladder work, and the script measures all three.

The next guess. Between rungs the flight-time guess is scaled by $t_f \leftarrow t_f (T_{\text{prev}}/T)^{0.6}$, an empirical law for this band. On the default run it predicts the next answer to about 1% on most rungs; the largest printed error is 2.1%, on the last step, $0.75 \rightarrow 0.5$ N (guess 0.7416 ND, answer 0.7575 ND). The script prints the ratio on every rung, so “starts next to its answer” is a number.

The screen. A rung that lands far from where the scaling predicted has landed somewhere the guess did not describe — and it can look perfect while doing so: converged, defect 10^{-14} , clear of the Moon. Comparing the answer with the guess that produced it is the only cheap thing that catches it. It is a plausibility heuristic, not a branch detector: a sharp turn on the same branch can trip it, and a jump to a nearby-time branch can pass it. It applies to steps only, never to the cold top rung, whose guess has no branch behind it (the true 15 N answer is $41\times$ below the 4.0 ND cold guess).

The throttle. The solver leaves it free; every rung is checked to have saturated, on both sides of 1, over the node and midpoint throttle histories the solver returns (normally both; the predicate does not separately require both arrays). Minimum time is all-burn, and a rung whose throttle dipped would not be the problem the shooting solver solves next.

section 3: the inter-rung guess

```
if ~isempty(Tprev), seedTf = seedTf*(Tprev/TN)^dladder.tfExp; end
```

section 3: the band screen, steps only

```
band = oR.tf/seedTf;
inBand = isempty(Tprev) || (band <= dladder.tfBand && band >= 1/dladder.tfBand);
clearR = radR >= num.clearKm;
okR = feasR && clearR && inBand;
```

Rigor

Delta-V falls down the ladder, and why. At fixed exhaust speed and initial mass the all-burn mass law is $m(t) = 1 - T_{\text{nd}}t/c_{\text{nd}}$, so with $a = T_{\text{nd}}t_f/c_{\text{nd}}$ the propellant fraction used is a and the ideal

$$\Delta V_{\text{nd}} = c_{\text{nd}} \ln \frac{1}{m_f} = -c_{\text{nd}} \ln(1 - a), \quad \frac{d\Delta V_{\text{nd}}}{da} = \frac{c_{\text{nd}}}{1 - a} > 0 \quad (0 \leq a < 1).$$

If $t_f \propto T^{-0.6}$ then $T t_f \propto T^{0.4}$ falls as thrust falls, so a falls and so do both the propellant and the ideal ΔV : lower thrust, longer trip, less propellant. The script prints the dimensional value $\Delta V_{\text{nd}} l^*/t^*$ in km/s. Measured $15 \rightarrow 0.5$ N: $4.22 \rightarrow 0.996$ km/s, mass used $22.3\% \rightarrow 5.8\%$.

The predicate. Every direct solve — cold, ladder, Sundman re-solve, hunt — goes through one *core* predicate, and each stage may add its own screens on top (the ladder adds the lunar

floor and the band; the hunt adds the floor; the default lottery is unconstrained). Required diagnostics default to NaN, not 0, so a missing one fails; the state array must be finite; the throttle history must be finite and satisfy the strict inequalities $1 - \epsilon < q < 1 + \epsilon$. The first failed check is named, and the reasons are reported in an order that never calls an unconverged iterate a branch change.

section 3: ideal delta-V from the mass actually used

```
dv = cnd*log(1/oR.mf)*lStar/tStar; % ideal dV from the mass actually used
```

local function directOK: one predicate for every direct solve

```
fin = @(v) isnumeric(v) && isscalar(v) && isfinite(v) && isreal(v);
thrAll = [];
if isfield(o, 'U') && ~isempty(o.U) && size(o.U, 1) >= 4, thrAll = [thrAll, o.U(4, :)]; end
if isfield(o, 'Um') && ~isempty(o.Um) && size(o.Um, 1) >= 4, thrAll = [thrAll, o.Um(4, :)]; end
thrLo = NaN; thrHi = NaN;
if ~isempty(thrAll) && all(isfinite(thrAll)), thrLo = min(thrAll); thrHi = max(thrAll); end
checks = { ...
    'not converged', ~ (isfield(o, 'success') && isscalar(o.success) && o.success); ...
    'non-finite fields', ~ (fin(gvd(o, 'tf', NaN)) && fin(gvd(o, 'mf', NaN)) && fin(gvd(o, 'maxDefect', NaN)) && ...
    fin(gvd(o, 'termErr', NaN)) && fin(gvd(o, 'maxUnit', NaN))); ...
    'states not finite', ~ (isfield(o, 'X') && ~isempty(o.X) && all(isfinite(o.X(:)))); ...
    'collocation defect', ~ (gvd(o, 'maxDefect', NaN) < tol.defect); ...
    'interpolation residual', ~ (fin(gvd(o, 'maxInterp', NaN)) && gvd(o, 'maxInterp', NaN) < tol.interp); ...
    'lifted t_f spread', ~ (fin(gvd(o, 'tfSpread', NaN)) && gvd(o, 'tfSpread', NaN) < tol.tfSpread); ...
    'terminal residual', ~ (gvd(o, 'termErr', NaN) < tol.termErr); ...
    '|u| = 1', ~ (gvd(o, 'maxUnit', NaN) < tol.unit); ...
    'throttle history', ~ (isfinite(thrLo) && isfinite(thrHi)); ...
    'throttle not saturated', ~ (1 - thrLo < tol.thrMin); ...
    'throttle above one', ~ (thrHi - 1 < tol.thrMin); ...
    'mass fraction', ~ (gvd(o, 'mf', NaN) > 0 && gvd(o, 'mf', NaN) <= 1); ...
    't_f not positive', ~ (gvd(o, 'tf', NaN) > 0);
bad = find([checks{:, 2}], 1);
ok = isempty(bad);
if ok, why = ''; else, why = checks{bad, 1}; end
```

Measured

Rung ratios matter. The first version of the ladder stepped [15, 10, 5, 2, 1, 0.5]. The $1 \rightarrow 0.5$ N step (ratio 0.5) returned a converged solution with defect 3.9×10^{-14} , a node clearance above the floor and $\|u\| = 1$ at $t_f = 54.7$ d and $\Delta V = 46.8$ km/s: 16× its own guess, 94% of the mass burned, a very different angular excursion. The ladder accepted it, and the handoff then ground on a 54-day arc with integrator failures. The shipped rung set steps by about 0.75 and does not jump; the band screen refuses the 16× step. On the default run all eleven rungs accept, the worst throttle slack is 1.3×10^{-6} , and t_f goes $0.432 \rightarrow 3.358$ d.

7 §4 — The handoff: where costates come from

ELI5

The big optimiser never heard of costates. But to enforce “the trajectory obeys physics” at every step it keeps a set of bookkeeping numbers — how much the trip time would improve if it were allowed to cheat at that step. Those bookkeeping numbers contain approximations to the steering costates. After applying the right sign, scale and time-station mapping, lay them along the trip at 25 checkpoints and hand that to the careful solver: it now has a starting trajectory with the steering numbers filled in.

Intuition

Three subtleties turn that from a slogan into working code, and the script checks two of them and says plainly that it cannot check the third. *Station*: in this transcription’s covector mapping the defect multipliers are associated with segment *midpoints*; associating them with nodes misplaces every harvested costate sample by half an interval, which degrades the seed and biases any direct-versus-indirect costate comparison. (A shooting solve that then converges still satisfies its own boundary-value problem; the damage is to the seed and the comparison.) *Sign*: the convention depends on how the NLP was written, so it is settled by a vote — the minimum principle requires the applied direction to oppose λ_v at every station, and the sign that satisfies that at the most stations wins (100% on the default run). *Scale*: with t_f as the objective, the multiplier of the time equation must come out $\lambda_t = +1$.

That last row exists only in the Sundman chart, where time is itself a state. The ladder ran in plain time, as the shipped catalog did, so the script *tries* to re-solve the bottom rung once with Sundman on, warm from the plain-time rung. It harvests from the re-solve only if it passes the core predicate and two consistency screens — t_f and node radius within 1% of the plain-time rung — and otherwise falls back to the plain-time solution, where H3 is not applicable. Passing the screens permits the handoff; it does not prove the two discretisations represent the same continuous root. On the default run it buys $\lambda_t = 1.000000$. What a constant row *cannot* see is a half-step station shift: midpoint association is a rule `duals_to_costates` applies, not a thing this number proves.

section 4: Sundman re-solve, time and radius consistency screens

```
dSun = dLad; dSun.sundman = true;
oH8 = solveDirect(rv0, rvf, TndB, cndB, muStar, dladder.N, oPrev.X, oPrev.U, oPrev.tf, dSun);
[feasS, whyS] = directOK(oH8, tol);
% the re-solve is accepted as the SAME solution as the plain-time rung on
% two enforced measurements: t_f agrees to tol.same and the node-sampled
% lunar radius agrees to tol.same -- a re-mesh moves both slightly. An
% aligned trajectory comparison is not made; these two are what is claimed.
sameTf = abs(oH8.tf - oPrev.tf)/oPrev.tf < tol.same;
sameRad = abs(oH8.altMinKm - oPrev.altMinKm)/max(oPrev.altMinKm + rMoonKm, 1) < tol.same;
if ~feasS, sunNote = ['Sundman re-solve refused: ' whyS]; oH8 = [];
elseif ~sameTf, sunNote = sprintf('Sundman re-solve moved t_f by %.1f%%: not the same solution', ...
    100*abs(oH8.tf - oPrev.tf)/oPrev.tf); oH8 = [];
elseif ~sameRad, sunNote = sprintf('Sundman re-solve moved the node radius by %.0f km: not the same
    solution', ...
    abs(oH8.altMinKm - oPrev.altMinKm)); oH8 = [];
```

section 4: harvest, then shoot

```
[seedB, dgB] = harvest_ms_seed(oHarv, num.K);
tH = tic;
[okB, zB, itB] = fenced(pool, @ms_tfmin, 2, num.shootSec + 90, rv0, rvf(1:6), seedB, TndB, cndB, muStar,
    ...
    struct('tolR', tol.Rsolve, 'wallSec', num.shootSec, 'maxIter', 100));
```

Rigor

Sundman and λ_t . On $\tau \in [0, 1]$, with $\kappa(\mathbf{x}) = \rho^p$ (ρ the lunar distance) and primes for τ -derivatives, write the minimum-time problem in *Mayer* form, with the clock as a state and no running cost:

$$\min t(1), \quad \mathbf{x}' = S\kappa(\mathbf{x}) \mathbf{f}(\mathbf{x}, \mathbf{u}, q), \quad t' = S\kappa(\mathbf{x}), \quad t(0) = 0.$$

With the normal minimising convention, $\mathcal{H} = S\kappa(\mathbf{x})(\lambda_x^\top \mathbf{f} + \lambda_t)$. Nothing depends explicitly on the clock, so $\lambda_t' = -\partial \mathcal{H} / \partial t = 0$, and the Mayer terminal condition gives $\lambda_t(1) = \partial t(1) / \partial t(1) = 1$. Hence $\lambda_t \equiv 1$ and $\mathcal{H} = S\kappa H$. (Carrying the running cost of the original form as well would count time twice; and under the sign-reversed maximum convention the constant is -1 — the $+1$ is tied to the convention.) The eighth multiplier row of the Sundman NLP is a discretisation of that constant, so its value is a free check on the sign and scale of the duals-to-costates mapping.

Multiple shooting. Unknowns $p = [\lambda(0); \mathbf{Y}_2; \dots; \mathbf{Y}_K; t_f]$ with $\mathbf{Y}_k \in \mathbb{R}^{14}$ the junction starts ($\mathbf{Y}_1$'s state part is fixed), $7 + 14(K - 1) + 1$ scalars; equations

$$\mathbf{Y}_{k+1} - \Phi_k(\mathbf{Y}_k) = 0 \quad (k = 1..K - 1), \quad \psi(\Phi_K(\mathbf{Y}_K)) = \begin{bmatrix} \mathbf{r}(t_f) - \mathbf{r}_f \\ \mathbf{v}(t_f) - \mathbf{v}_f \\ \lambda_m(t_f) \\ H(t_f) \end{bmatrix} = 0,$$

$14(K - 1) + 8$ of them, so $14K - 6$ unknowns and equations: 330 each at $K = 24$. Square does not mean nonsingular. The continuity rows have block-bidiagonal junction structure, bordered by the free- t_f column and the terminal rows; each block is an STM over one short segment rather than the whole arc, which reduces the long-shot sensitivity amplification, but the full Jacobian can still be ill-conditioned or singular. `ms_tfmin` returns `info.Y` as the $14 \times K$ junction *starts* and `info.tGrid` as $K + 1$ times; the endpoint column is never stored and no consumer reads it (the library headers that say $K + 1$ are what is wrong). The harvested seed, by contrast, is $14 \times (K + 1)$: different objects, not contradictory counts.

Gates. H1: shooting residual under `tol.R`. H2: sign vote $\geq 90\%$. H3 has *three* states — NOT APPLICABLE in plain time (kept out of the pass count), PASS/FAIL in the Sundman chart, where a missing or non-finite λ_t , or a mapping validity flag that is not true, fails. Missing evidence never reads as agreement.

section 4: H3 with three states

```
h3na = isempty(oH8);
if h3na, h3 = false;
else, h3 = isfinite(lamTB) && abs(lamTB - 1) < 1e-2 && islogical(lamTOKB) && isscalar(lamTOKB) &&
    lamTOKB;
```

Measured

Default run: the Sundman re-solve differs from the plain-time rung in t_f by 3.80×10^{-8} relative, and both minimum node radii round to 6414 km. The minimum node radius is 4514 km above the floor — strong sampled evidence that the clearance constraint is inactive on the harvested arc, which is what permits an unconstrained covector handoff, though not a continuous-clearance proof; the shooting solve then checks the unconstrained problem independently. Sign vote 100%; $\lambda_t = 1.000000$ with the mapping's own flag true; the handoff reached $\|R\| = 4.59 \times 10^{-12}$, below the requested 10^{-11} , in one reported iteration.

8 §5 — The indirect ladder

ELI5

Below about half a newton the big optimiser starts losing the thread — the trip winds more and the good answers get harder to find. The careful solver cares less, because it only ever looks at 24 short pieces. So the walk continues with it. And the starting point for the next rung needs no computing at all: the 24 checkpoints of the last answer *are* the starting point, with their clock stretched to the new guess and the fuel column redone for the new engine.

Intuition

Why no propagation? The banked junction starts are the states and costates at the normalised breakpoints σ_k (uniform, k/K , in this pipeline). The next rung uses the same K and the same normalised grid, so the seed is the banked array with its time grid scaled to the new t_f guess and its mass row rebuilt from the all-burn law for the new engine. The seed need not satisfy the new dynamics before shooting — that is the shooter's job. Re-flying the whole arc from $\lambda(0)$ would bring back the amplification multiple shooting exists to avoid, and interpolating a flight would perturb the very numbers the previous solve converged.

Why a sweep of exponents? Past a winding wall the next solution has a different angular excursion, and a different exponent is what puts the guess near it. The *fastest accepted* candidate across the sweep is kept. Say what that is: a small search over guesses at each rung, not branch-preserving continuation. It is the campaign's own rule, it is what got the recorded chain past a wall, and it can switch branches — which is why the turn count is printed beside every rung.

Why a stall is a finding. A rung with no accepted candidate is a statement about *this search policy* — these ratios, exponents, budget and seed cap — on this cell. It is not, by itself, a proof that no root exists below. Two in a row stop the walk, and every attempt is recorded with its reason.

section 5: the next seed, without propagating anything

```
Yg = itCur.Y; Yg(1:7, 1) = [rv0(:); 1];
tGs = sigK*tfGuess;
Yg(7, :) = 1 - Tnd*tGs(1:size(Yg, 2))/cnd;
capShoot = min(num.shootSec, left);
[okRun, zt, it] = fenced(pool, @ms_tfmin, 2, capShoot + 90, ...
    rv0, rvf(1:6), struct('tf', tfGuess, 'tGrid', tGs, 'Y', Yg), ...
    Tnd, cnd, muStar, struct('tolR', tol.Rsolve, 'wallSec', capShoot));
```

Rigor

With $\sigma_k = t_k/t_f$ the fixed normalised breakpoints and t_f^g the guess,

$$t_k^{\text{seed}} = \sigma_k t_f^g, \quad \mathbf{Y}_{7,k}^{\text{seed}} = 1 - \frac{T_{\text{nd}} t_k^{\text{seed}}}{c_{\text{nd}}}, \quad t_f^g = t_f^{\text{prev}} \left(\frac{T_{\text{prev}}}{T} \right)^e, \quad e \in \{1.0, 1.3, 1.7, 2.2, 3.0\},$$

with e irrelevant (one attempt) on an Isp-only step. The seed-propellant policy skips a guess with $t_f^g > 0.6 c_{\text{nd}}/T_{\text{nd}}$ — a choice about which seeds to try that keeps 40% of the mass, *not* the exhaustion bound $c_{\text{nd}}/T_{\text{nd}}$ — and records the skip.

Acceptance of a rung is three-fold and fail-closed: the shot converged ($\|R\| < \text{tol.R}$, finite z , $t_f > 0$); the flown witness from z alone reached t_f with positive mass and a finite $n \times 14$ flight; and it landed inside the position *and* velocity gates (100 km, 10 m/s). “Accepted” means a usable continuation seed; physical validation is Section 9’s job.

The angular excursion printed as “turns” is, along the samples of the witness flight in the rotating frame,

$$\theta_i = \text{unwrap} \left[\text{atan2} \left(y_i, x_i - (1 - \mu^*) \right) \right], \quad \text{turns} = \frac{1}{2\pi} \sum_i |\theta_{i+1} - \theta_i|.$$

It is a *sampled estimate* of the total variation of the Moon-centred azimuth, so reversals add. Unwrapping assumes adequate angular sampling and cannot recover reversals between samples, and the azimuth needs a nonzero projected Moon-centred radius. It is a diagnostic, not a winding number.

Budget. One deadline per rung (600 s of requested work): the remaining time is recomputed before the shoot and before the witness, and no stage starts with under 10 s left. It is not a strict wall-clock cap: the shooting fence allows a further 90 s of cancellation grace, and any overrun is charged at the next remaining-budget calculation. Direct NLP solves run under CPU caps, not these fences. A fenced call that returns *not ok* is “timed out or the worker errored” — `run_capped` cannot tell them apart. Without a parallel pool the external fences disappear entirely: `fenced` calls the function directly, only the solvers’ own internal limits remain, and certification is skipped unless `num.allowUnfenced` is set.

section 5: the witness gate, fail-closed

```
reached = okW && isnumeric(yFly) && size(yFly, 2) == 14 && size(yFly, 1) == numel(tw) && ...
    ~isempty(tw) && all(isfinite(yFly(:))) && all(isfinite(tw(:))) && ...
    abs(tw(end) - zt(8)) < 1e-9*max(1, zt(8)) && yFly(end, 7) > 0;
if ~reached, A(end+1) = attempt(tfExp, tfGuess, 'witness flight did not reach t_f', it.normR, NaN, NaN, toc
    (tA)); continue, end
mk = norm(yFly(end, 1:3).' - rvf(1:3))*lStar;
mv = norm(yFly(end, 4:6).' - rvf(4:6))*lStar/tStar*1000;
if ~(isfinite(mk) && mk < num.gateKm && isfinite(mv) && mv < num.gateVms)
    A(end+1) = attempt(tfExp, tfGuess, 'flown miss outside the gates', it.normR, mk, mv, toc(tA));
    continue
end
A(end+1) = attempt(tfExp, tfGuess, 'accepted', it.normR, mk, mv, toc(tA));
```

section 5: the angular-excursion diagnostic

```
ang = unwrap(atan2(best.yFly(:, 2), best.yFly(:, 1) - (1 - muStar)));
turns = sum(abs(diff(ang)))/(2*pi);
```

Measured

Default run: $0.375 \rightarrow 0.12$ N at $I_{sp} = 1710$ s (7 rungs), the Isp stage $1710 \rightarrow 1400 \rightarrow 1150 \rightarrow 900$ s at 0.12 N (3 rungs in about 3 s, t_f falling $11.119 \rightarrow 10.606$ d), then 0.11 and 0.10 N: every one of the five exponents *unconverged*, no fence event, no seed-policy skip. Turns climbed $0.81 \rightarrow 1.31$. Worst accepted witness miss 1.8×10^{-4} km. Three reported routes from this anchor cell stalled at 0.12 , 0.105 and 0.11 N (coarse ratios; finer ratios; finer ratios with the Isp stage). A fourth, the campaign's own route, started from the sheet's fastest 0.5 N cell rather than this one and reached 0.09 N before stalling at 0.067 N. Where you start the deep walk is a choice, not a detail.

9 §6 — Certification, and adopting the polished root

ELI5

Now hand the trip to the inspector. It re-solves it to higher precision, flies it from the initial numbers alone, checks the conditions of the minimum principle along that flight, asks a second, independent solver to agree, and runs tests designed to tell a local minimum from a saddle. If it passes, the inspector's re-solved version is the one we keep. A pass supports *local* minimality under those tests' assumptions; it is not a proof that no faster trip exists anywhere.

Intuition

A certificate applies to a particular polished trajectory and to the options the certifier actually consumed. So two interfaces have to stay consistent.

The options. The certifier reads the initial mass, the arrival gates and the lunar floor from *its own options*, not from the problem struct. Setting them only in the problem struct does nothing; editing them in script §0 without forwarding would silently leave certification on its defaults. They are forwarded, and printed on the certify line.

The time grid. The certifier polishes and returns the polished z and junction states, but no time grid. The shooter keeps the *normalised* breakpoints fixed and re-solves t_f , so the polished states belong to the polished t_f . Pairing them with the pre-polish grid would leave the record and the hunt's warm start inconsistent by the polish's change in t_f on every segment — even though the certificate itself was valid.

section 6: forward what certify_root reads, and the poolless policy

```
certOpts = struct('pool', pool, 'wallSec', num.certSec, 'sA', phase.sA, 'sD', phase.sD, ...
                  'm0kg', op.m0kg, 'gateKm', num.gateKm, 'gateVms', num.gateVms, 'moonKmMin', num.clearKm,
                  ...
                  'allowUnfenced', num.allowUnfenced);
if isempty(pool) && ~num.allowUnfenced
    % the poolless policy, stated in section 0: no fence, no certificate
```

```

C = struct('ok', false, 'reason', 'not certified: no parallel pool and num.allowUnfenced is false', ...
          'tfDays', day(zCur(8)), 'flyKm', NaN, 'flyVms', NaN, 'z', zCur(:), 'Y', itCur.Y);
else
C = certify_root(seedR, rv0, rvf, B, certOpts);

```

section 6: adopt the polished root WITH its grid

```

tfPre = zCur(8);
zCur = C.z(:);
itCur.Y = C.Y; itCur.tGrid = sigK*zCur(8); itCur.normR = gvd(C, 'normR', NaN);

```

Rigor

With $t_f^{\text{cert}} := C.z(8)$ the polished flight time, $t_k^{\text{cert}} = \sigma_k t_f^{\text{cert}}$, with σ the normalised grid preserved from the walk; the shapes are asserted before adoption and the relative change in t_f is printed (it reads 0 when the walk’s root was already polished, which is the honest number). Without a parallel pool the certifier refuses to run unfenced unless told to; the script’s policy knob `num.allowUnfenced` (default false) makes that explicit and records “not certified: no pool” rather than aborting after an hour.

The gate stack itself — polish, flown arrival in position and velocity, pointwise PMP checks, the foreign-solver witness, the free-time conjugate test, the hypothesis gates and the H6 validity condition — is `transfer_study_guide.tex`’s subject. Here only its verdict is used, and outcome B is decided *after* it: SUPPORTED means a certified root at the target engine; “reached numerically” is its own state.

section 6: outcome B, decided after certification

```

if b2 && C.ok, outcomeB = sprintf('SUPPORTED: a certified root at the target engine, walked from a
    cold %g N start', dladder.rungs(1, 1));
elseif b2, outcomeB = 'REACHED NUMERICALLY: the target engine, but the root there did not certify
    ';
elseif iladder.run, outcomeB = sprintf('PARTIAL: reached %g N / %g s of %g N / %g s%s', Tcur, ispCur, op.
    thrustN, op.ispS, ...
    tern(C.ok, ', certified there', ', not certified there'));
else, outcomeB = sprintf('PARTIAL: direct ladder only, %g N / %g s%s', Tcur, ispCur, ...
    tern(C.ok, ', certified there', ', not certified there'));
end

```

10 §7 — The basin hunt

ELI5

Now that we hold one trip, go looking for others. Ask the big optimiser for a trip to a *different* arrival point, starting from the trip we have. It does not know which family it came from, so it may hand back another — sometimes a faster one. That is how the fast2, direct18 and direct11 families in the library were found.

Intuition

What a probe measures, exactly. Each probe is a *different* boundary-value problem (the arrival state moved), so its t_f is not comparable to the source's as a basin test: one family's t_f varies with phase by far more than a few percent, and two families can share a t_f . A probe yields a *candidate*: a converged transfer at phase s_A , some percent faster or slower than the source *at its own phase*. Turning a candidate into a basin verdict needs a same-phase baseline — continue the held family to s_A with `arclength_arrival` and compare states and costates there. That is what FINDINGS 61, 63 and 68 did. Every probe starts from the *same* held root, not from the previous probe's result. A probe that passes the core predicate and the floor keeps its direct solution arrays; a seed is harvested from it when usable multipliers came back, and those seeds are banked for later shooting and certification.

Rigor

The warm start needs dense samples on $N + 1$ nodes, which the banked junctions alone are not, so here — and only here — the source root is re-flown *segment by segment* from its junction starts. Segment k lasts $t_f(\sigma_{k+1} - \sigma_k) - t_f/K$ on this pipeline's uniform grid — and is propagated independently from its banked start. At each interior seam the *banked start* of the next segment is kept and the previous segment's propagated endpoint dropped, so a resample at a junction recovers the banked state exactly; the worst *position* seam mismatch is returned in km rather than hidden (2.7×10^{-8} on the default run). Duplicate time samples are removed, the primer norm is required finite and positive, and the direction guess is $\mathbf{u}_0 = -\lambda_v / \|\lambda_v\|$ with throttle 1.

local function flyFromJunctions: segment by segment, the banked start wins the seam

```
for k = 1:K
    dt = tG(k+1) - tG(k);
    [ts, ys] = pumpkyn.cr3bp.tfMinProp(dt, Y(:, k), Tnd, cnd, mu);
    assert(isnumeric(ys) && size(ys, 2) == 14 && size(ys, 1) == numel(ts) && all(isfinite(ys(:))) && ...
        abs(ts(end) - dt) < 1e-9*max(1, dt), 'flyFromJunctions:segment', 'segment %d did not complete',
        k);
    if k < K
        seamKm = max(seamKm, norm(ys(end, 1:3).' - Y(1:3, k+1))*1StarKm);
        ts = ts(1:end-1); ys = ys(1:end-1, :); % the banked start of k+1 wins the seam
    end
    tj = [tj; tG(k) + ts(:)];
    yj = [yj; ys];
end
```

section 7: the direct warm start from the re-flown source

```
sN = linspace(0, tjH(end), dladder.N + 1);
X0h = interp1(tjH, yjH(:, 1:7), sN, 'spline').';
LVh = interp1(tjH, yjH(:, 11:13), sN, 'spline');
nLV = vecnorm(LVh, 2, 2);
assert(all(isfinite(nLV)) && all(nLV > 1e-12), 'the primer vanishes or is not finite on the source flight')
;
U0h = [(-LVh ./ nLV).'; ones(1, dladder.N + 1)];
```

Rigor

Three counts are kept apart for each probe: did the solver converge, was the solution accepted (`directOK` and the floor), was a seed harvested from it. An accepted solution whose multipliers did not come back keeps its numbers ($\mathbf{X}, \mathbf{U}, t_f$) so “anchor it” stays possible by re-solving. A capped probe is reported as “solver did not converge” and nothing more — a lottery ticket lands on either side of its budget from run to run (the $s_A = 0.7837$ probe converged in 61 s on one run and hit its 300 s CPU cap on the next), and the script no longer reads that as anything about basins.

section 7: the reading for each probe, in order of precedence

```
if ~oH.success, rd = ['solver did not converge: ' oH.ipoptStatus];
elseif ~feasH, rd = ['converged but refused: ' whyH];
elseif ~clearH, rd = sprintf('converged but refused: node radius %.0f km under the %g km floor', radH,
    num.clearKm);
elseif rel < 0, rd = sprintf('candidate, %.1f%% FASTER than the source at its own phase -- worth a same-
    phase baseline', -100*rel);
else, rd = sprintf('candidate, %.1f%% slower than the source at its own phase', 100*rel);
end
if okH && ~harvOK, rd = [rd ' (NO seed harvested: multipliers unavailable; direct solution kept)']; end
```

Measured

Default run, from the certified 0.12N root at $s_A = 0.0754$ ($t_f = 10.606$ d): $s_A = 0.3254 \rightarrow 11.240$ d (+6.0%, candidate); $s_A = 0.5754 \rightarrow 9.081$ d (−14.4%, candidate, banked); $s_A = 0.7837 \rightarrow$ CPU cap. Outcome C: CANDIDATES, 2 accepted, 1 faster, 2 banked; basin identity not established here.

11 §8 — The self-check: gates and outcomes are different things

ELI5

The script grades itself twice. First: did the machinery work? If the ladder produced nothing, or the harvest could not be shot to a root, there is no usable root for the rest of the walk, and the script stops right there. Second: what did the three experiments *show*? A search that found nothing is a result, not a broken machine.

Rigor

Among the named gates, B1 (the direct ladder produced a rung) and H1 (the harvest shot to a root) are required to continue, and each is asserted when its stage ends, not at the end of the run. Other validation assertions — orbit closure, usable multipliers, a malformed passing certificate — and unhandled helper errors can also abort. A failed handoff does not invalidate the cold lottery already recorded. B1b, H2, H3, B2 and C1 are reported, never fatal; H3 is removed from the table when not applicable; the final table summarises the gates execution reached. Outcomes take the values SUPPORTED, PARTIAL, REACHED NUMERICALLY, CANDIDATES, NOT OBSERVED, INCONCLUSIVE, and NOT RUN for a stage switched off. The record — configuration, tolerances, every rung and probe with its reason,

the certificate, the value-only certifier options, the gate table and the outcomes — is saved; a failed save warns, which is not a guarantee.

section 8: the gate table

```
gateStatus = { ...
    'B1 direct ladder produced a rung', b1; ...
    'B1b direct ladder finished',      b1b; ...
    'H1 handoff shot to a root',       h1; ...
    'H2 harvest sign vote',            h2; ...
    'H3 harvest lambda_t',             h3; ...
    'B2 target engine reached',        b2; ...
    'C1 certified',                    C.ok};
if h3na, gateStatus(strcmp(gateStatus(:, 1), 'H3', 2), :) = []; end % not applicable is not a verdict
outcomes = {'A cold lottery', outcomeA; 'B ladder', outcomeB; 'C basin hunt', outcomeC};
```

12 What the script does not establish

Where it breaks / what is assumed

- That the lottery's converged rows are distinct *continuous* extremals. It shows mesh sensitivity of accepted discrete solutions.
- That the throttle is exactly saturated. The 10^{-3} gate is a seed-compatibility screen; the all-burn shoot and the certificate carry the claim.
- That the stall at 0.11 N is a property of the problem. It is a property of this search policy on this cell; arclength steps in thrust, rank and sensitivity of the shooting Jacobian and a budget study would be needed to say more.
- That any hunt candidate is another family. That needs a same-phase baseline.
- That a certified root at 0.12 N says anything about 0.07 N. Outcome B on the default run is PARTIAL, and says so.
- Anything about a capped probe except that it was capped.
- That any root found is the globally fastest transfer, or that all families have been enumerated.
- That the Sundman re-solve and the plain-time rung are the same continuous root; the two scalar screens only reject a gross move.
- That an accepted direct solution or a harvested hunt seed is a certified continuous extremal.
- That sampled clearance diagnostics and numerical integrations are error-enclosed continuous proofs.
- Anything beyond the specified CR3BP, ideal engine and mass model, and fixed end-point phases: not mission robustness, not a higher-fidelity ephemeris model.
- That the Hamiltonian argument of Section 1 holds where its assumptions fail — an active clearance constraint needs the constrained minimum principle.

13 Where these caveats came from

Most of the qualifications above were, at some point, a sentence in an earlier draft — of the script or of this guide — that claimed more than the computation earned. They are kept out of the

explanations so that each box explains rather than recounts, and gathered here because the failure modes are instructive.

- **Script review, round 1** (FINDINGS 76): the throttle was not checked at all; the certifier’s options were never forwarded; the hunt called flight-time differences across phases “another basin”; a sentence said ΔV rises down the ladder under a table showing it fall; the historical lottery table mixed altitude with radius.
- **Script review, round 2** (FINDINGS 77): the polished root was adopted with its pre-polish time grid; the poolless path could not finish; H3 read missing evidence as a pass; the reference was compared against problems it did not belong to; the ladder re-flew and re-interpolated a converged root instead of reusing its junctions.
- **Review of this guide:** the Hermite–Simpson equations lacked their time scaling; the turns formula lacked the unwrap; the handoff box quoted a smoke run as the default run; “to 1%” was 2.1% on the last step; the diagram drew the lottery as a side branch and hid the fallbacks and early fatal checks.

14 Summary

cold lottery	one cold solve per mesh; the two accepted rows disagree by 37% and neither matches the reference; motivates seeding or continuation
direct ladder	$15 \rightarrow 0.5\text{ N}$, guess $\propto T^{-0.6}$ to about 1% (worst 2.1%); band screen on steps; throttle checked both sides; ΔV falls because $Tt_f \propto T^{0.4}$
handoff	try a Sundman re-solve of the bottom rung so λ_t exists, else plain time; midpoint stations, sign vote, $\lambda_t = 1.000000$; $\ R\ < 10^{-11}$ in one iteration
indirect ladder	banked starts are the next seed, grid scaled, mass row rebuilt; exponent sweep is a search; per-attempt records; a stall is a policy finding
certify	options forwarded; polished root adopted with $t_k = \sigma_k t_f^{\text{cert}}$; poolless policy explicit; local, not global; outcome B decided after
basin hunt	re-fly from junctions, banked start wins the seam; candidates, not basins; seeds banked; capped is capped
self-check	B1, H1 asserted where their stages end; the rest reported; outcomes in their own words, including NOT RUN
